TaskFlow

AIMLEAP · FULL STACK DEVELOPER · TAKE-HOME ASSIGNMENT

TaskFlow: Mini Project Management Application

A full-stack project and task manager built with Next.js, NestJS and PostgreSQL, with authentication, projects, a drag-and-drop Kanban board, a live dashboard, tests, Docker and CI.

CANDIDATE
Mithlesh Dhakad

ROLE
Full Stack Engineer

EMAIL
mithleshdhakad89@gmail.com

SUBMITTED
16 October 2026

100%
mandatory requirements met

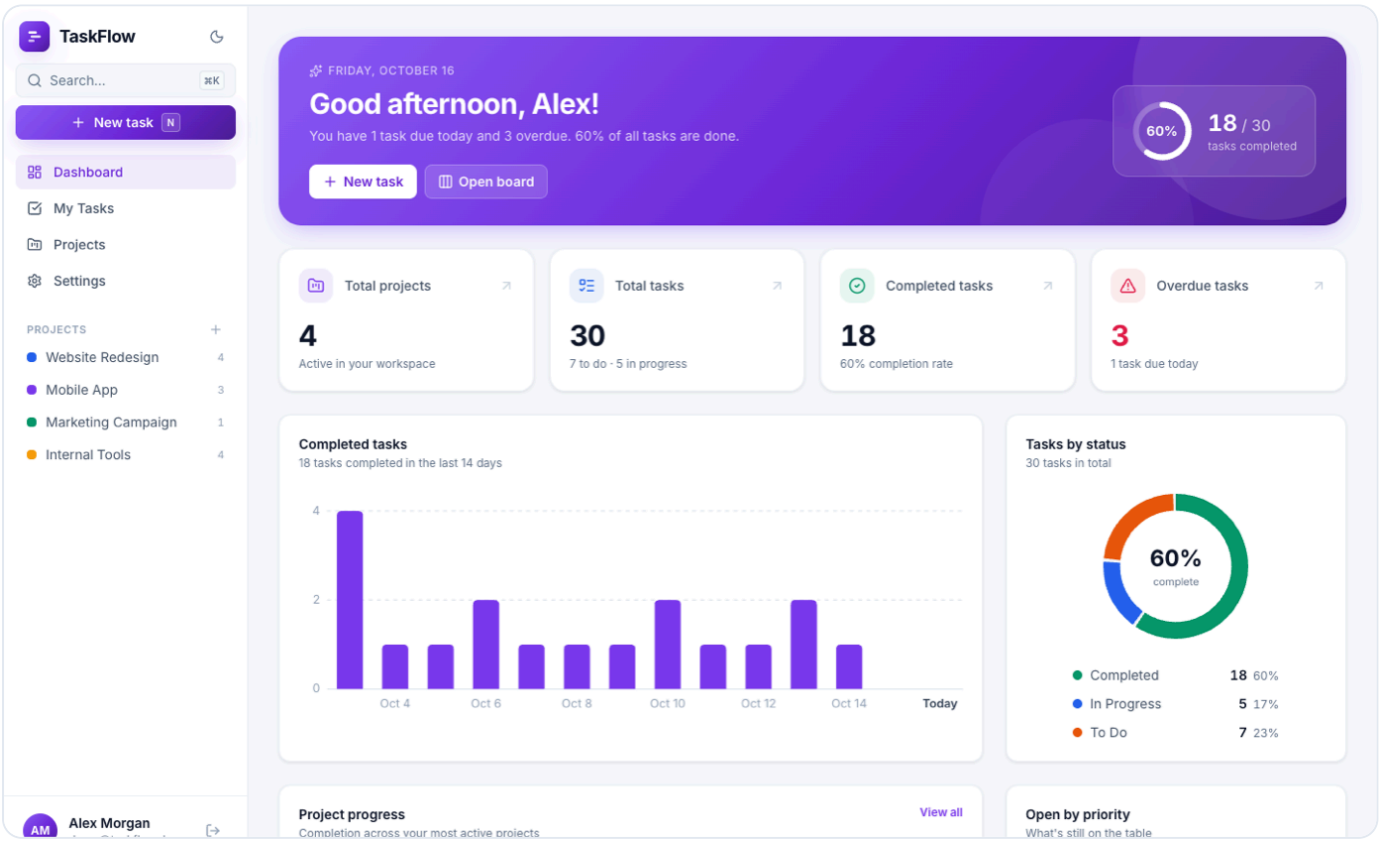
116
automated tests, all passing

3 / 3
GitHub Actions jobs green

1 cmd
`docker compose up --build`

SOURCE CODE REPOSITORY
`github.com/Mithlesh-dhakad1212/Task-Flow`

DEMO LOGIN
`demo@taskflow.dev / Password123!`



01 · OVERVIEW

Submission summary

Everything the brief asks for is implemented, tested and documented. The repository README and the docs listed below hold the full detail; this document is a guided summary.

Working end to end

Register, log in and out, projects, tasks, filters, search and dashboard, backed by PostgreSQL. Ownership is enforced on the server for every operation.

Production-minded

Rotating refresh tokens with reuse detection, consistent error envelope with request IDs, validation, rate limiting, indexes, migrations.

Reviewer-friendly

One-command Docker setup with seeded demo data, Swagger, Postman collection, architecture note and a line-by-line requirements matrix.

Submission requirements (brief §7)

#	REQUIRED	WHERE TO FIND IT	STATUS
1	**Source code**	GitHub repository: `backend/` (NestJS API) and `frontend/` (Next.js app)	✓
2	**README**	`README.md`: setup, environment variables, database configuration, run commands	✓
3	**Working application**	Clear local instructions: `docker compose up --build` → `localhost:3000` (see §2)	✓
4	**API documentation**	Swagger UI at `/api/docs` · `docs/openapi.json` · `docs/taskflow.postman_collection.json`	✓
5	**Database setup**	Prisma schema, SQL migrations and seed script; applied automatically by the Docker entrypoint	✓
6	**Tests + instructions**	116 tests (backend unit, backend e2e on real Postgres, frontend), README → *Testing*	✓
7	**Architecture note**	`docs/ARCHITECTURE.md`: stack, database design, authentication, key decisions	✓
–	**No secrets committed**	Only placeholder values in `*.example` files; secrets come from environment variables	✓

Technology stack

LAYER	CHOICE	WHY
Frontend	Next.js 15 (App Router), React 19, Tailwind CSS	Required by the brief; layouts give clean public vs. authenticated route groups
State management	TanStack Query + React Context + URL params	Caching, retries and optimistic updates for server data; Context only for auth, theme and overlays; filters live in the URL so views are shareable
Backend	NestJS 11 (Node.js 22)	Modules and dependency injection give a clear controller → service → data layering; guards, pipes and filters handle cross-cutting concerns
Database	PostgreSQL 16 + Prisma 6	Relational integrity (FKs, cascades, enums, CHECK constraints), type-safe queries, versioned migrations
Testing	Jest + Supertest, Vitest + Testing Library, Newman	Unit tests, integration tests against a real database, component tests and an API smoke test
Delivery	Docker, Docker Compose, GitHub Actions	Reproducible one-command setup; CI runs lint, type checks, tests and a full-stack smoke test

02 · GETTING STARTED

How to run the application

The fastest path needs only Docker. The API container applies migrations and seeds a demo workspace on start-up.

Option A: Docker (recommended)

```
# 1. Get the code
git clone https://github.com/Mithlesh-dhakad1212/Task-Flow.git
cd Task-Flow

# 2. Start PostgreSQL, the API and the web app
docker compose up --build
```

SERVICE	URL	NOTES
Web app	`http://localhost:3000`	Log in with `demo@taskflow.dev` / `Password123!` or register a new account
REST API	`http://localhost:4000/api`	Health check: `/api/health`
Swagger UI	`http://localhost:4000/api/docs`	Log in once and every endpoint is callable via "Try it out"
PostgreSQL	`localhost:5433`	User / password / database: `taskflow`. If port 5433 is busy: `DB_PORT=5544 docker compose up --build`

Option B: Local development

```
# Database (or point DATABASE_URL at any PostgreSQL 14+)
docker compose up -d db

# API → http://localhost:4000
cd backend && cp .env.example .env     # set JWT_ACCESS_SECRET (openssl rand -base64 48)
npm install && npm run db:deploy && npm run db:seed && npm run start:dev

# Web → http://localhost:3000
cd ../frontend && cp .env.example .env.local
npm install && npm run dev
```

Key environment variables

VARIABLE	PURPOSE
`DATABASE_URL`	PostgreSQL connection string (required)
`JWT_ACCESS_SECRET`	Signing secret for access tokens. Required, at least 32 characters; the API refuses to start otherwise
`JWT_ACCESS_TTL · REFRESH_TOKEN_TTL_DAYS`	Token lifetimes (defaults: 15 minutes · 7 days)
`CORS_ORIGINS · COOKIE_SECURE`	Allowed web origins; secure cookies when served over HTTPS
`RATE_LIMIT_GLOBAL · RATE_LIMIT_AUTH`	Requests per minute per IP (all routes · login/register/refresh)
`NEXT_PUBLIC_API_URL`	API base URL used by the browser (frontend)

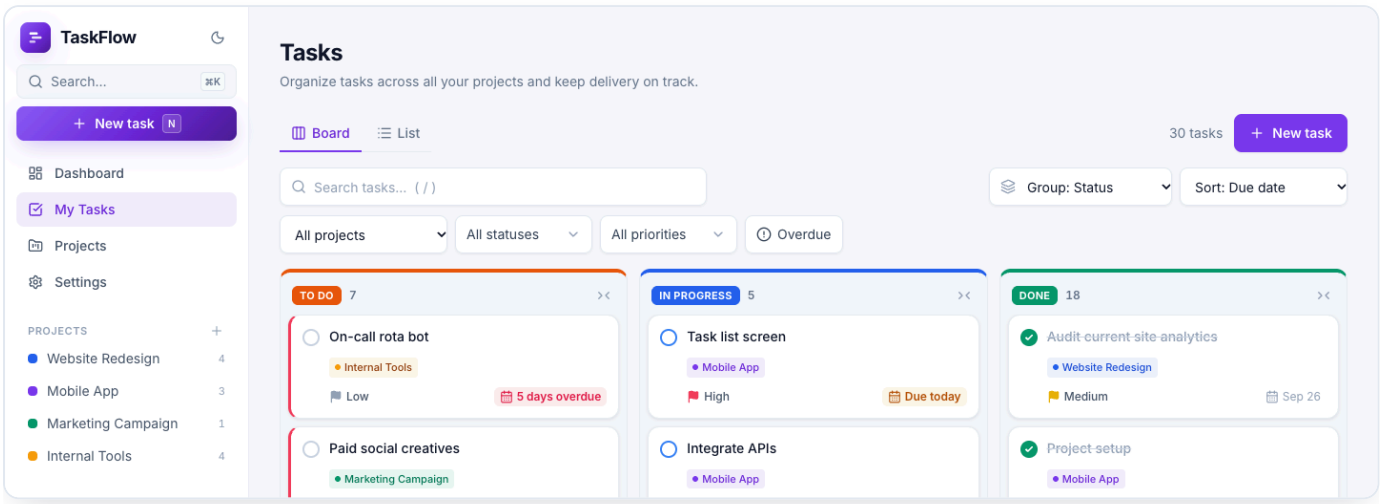
Running the tests

```
cd backend  && npm test            # 40 unit tests
cd backend  && npm run test:e2e    # 31 integration tests against PostgreSQL
cd frontend && npm test            # 45 component and unit tests
npx newman run docs/taskflow.postman_collection.json   # API smoke test (17 requests)
```

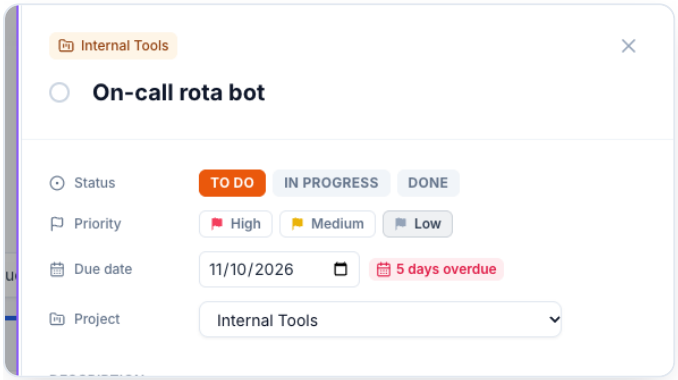
03 · PRODUCT WALKTHROUGH

What the application does

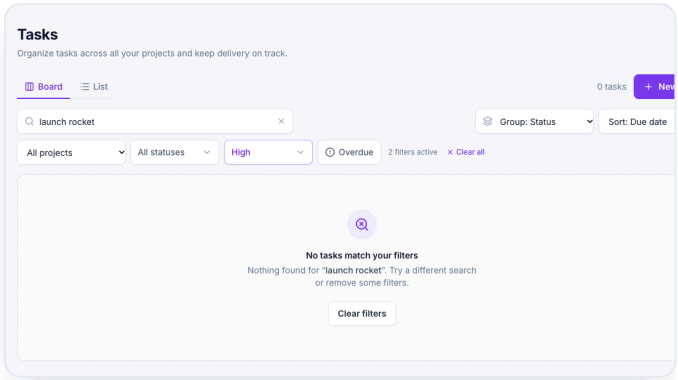
The reference screens in the brief were treated as inspiration; the UI is an original, ClickUp-inspired design with light and dark themes.



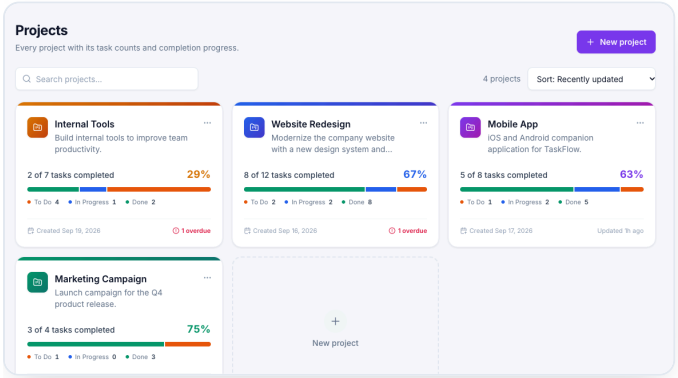
Kanban board. Drag cards between colour-coded columns to change status. The board can also be grouped by *priority* or *project*, and dropping a card then changes that field. Each column collapses and has an inline "+ Add task". Cards show a one-click complete toggle, priority flag, due-date chip (red when overdue) and project tag.



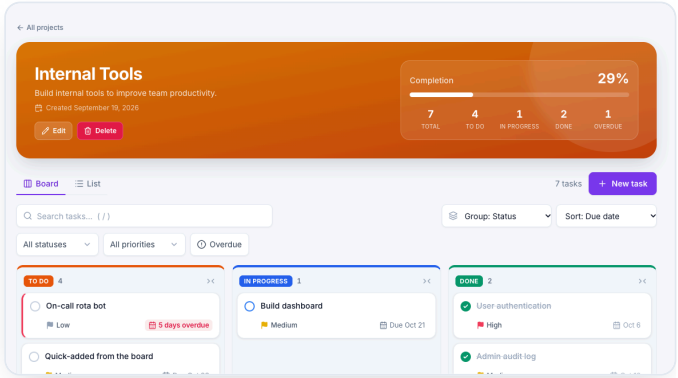
Task side panel. Edit the title, status, priority, due date, project and description inline with auto-save. Every task has a shareable link (`?task=<id>`).



Search + filters. Title search combined with status, priority, project and overdue filters, kept in the URL. A helpful empty state with "Clear filters" appears when nothing matches.



Projects. Create, edit and delete. Cards show name, description, creation date, task counts by status and completion progress.

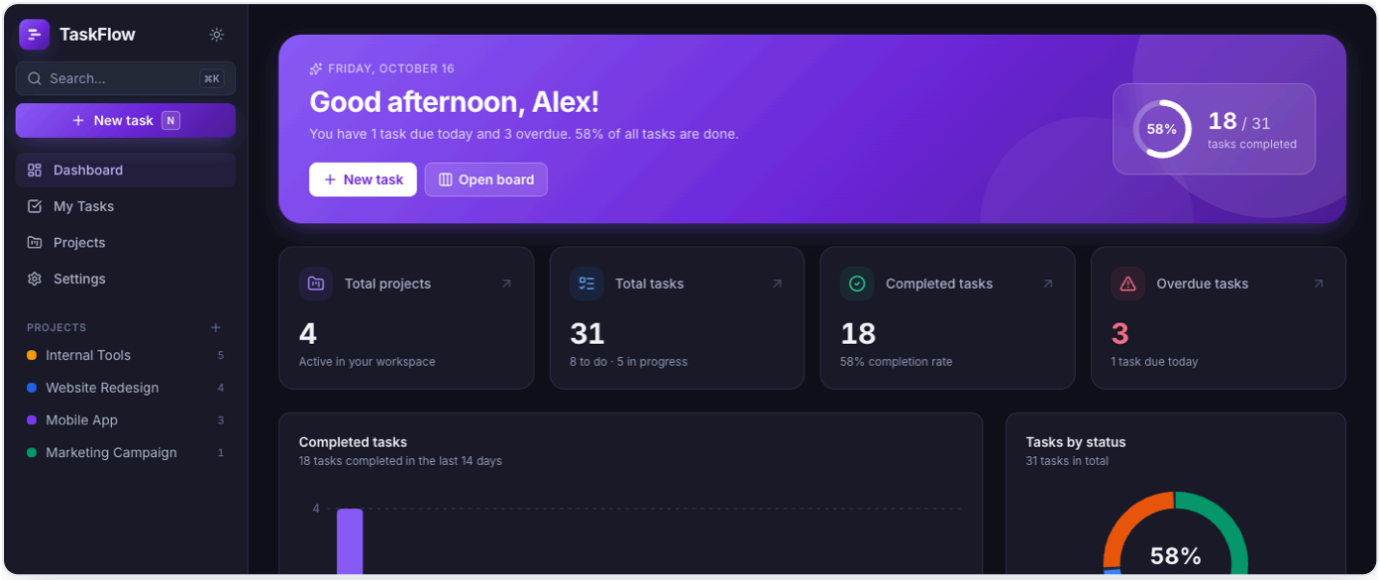


Project page. A banner with completion and stats, followed by the project's own board or list.

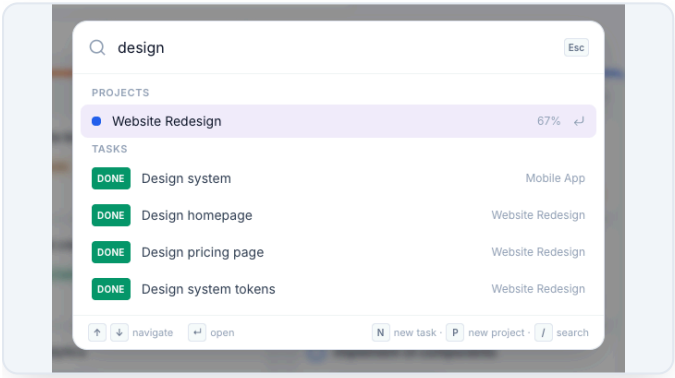
03 · PRODUCT WALKTHROUGH

Dashboard, themes and productivity

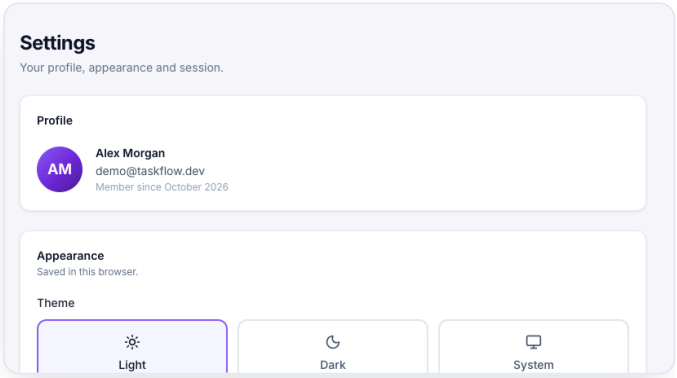
The dashboard covers the required totals and adds trend and priority views. All numbers are computed in SQL.



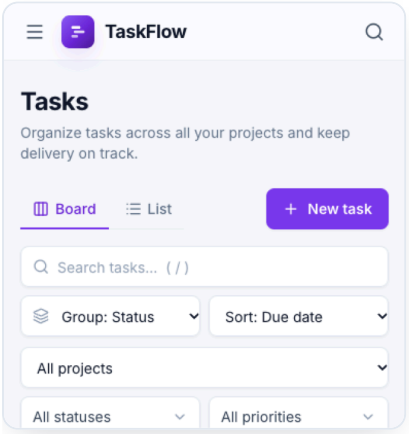
Dashboard (dark mode). Total projects, total tasks, completed and overdue tasks, a 14-day completed-tasks chart, tasks by status, open tasks by priority, project progress, and "up next" / overdue lists you can tick off in place. Overdue is computed in the user's own timezone.



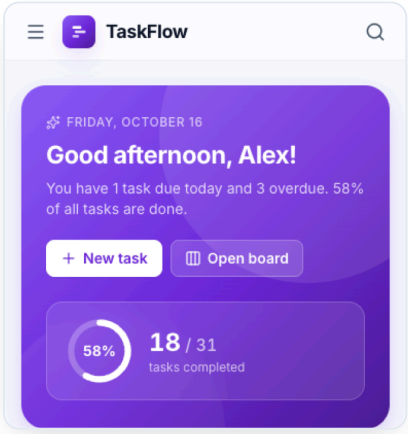
Command palette (⌘K / Ctrl+K). Search tasks and projects, create items, navigate and switch theme from the keyboard. Shortcuts: `N` new task, `P` new project, `/` search.



Appearance. Light / dark / system mode and five accent colours, applied before first paint (no flash). The status colours were checked for contrast and colour-blind safety in both themes.



Mobile board: columns scroll horizontally.



Mobile dashboard: responsive down to 360 px.

Thoughtful states everywhere

- Skeleton loaders while data loads
- Retryable error states with friendly messages
- Context-specific empty states with a next action
- Optimistic updates that roll back on failure
- Toasts for confirmations and errors
- Accessible dialogs: focus management, Escape, labelled fields

04 · REQUIREMENTS COVERAGE

Functional requirements

Every item is implemented and covered by automated tests; the full matrix with test references is in `docs/REQUIREMENTS.md`.

REQUIREMENT (BRIEF §2)	IMPLEMENTATION	
A · User authentication		
Register and log in with email and password	`POST /auth/register`, `/auth/login`; bcrypt-hashed passwords	✓
Protect project and task endpoints	Global JWT guard; every route is protected unless explicitly public	✓
Users access only their own projects and tasks	Owner filter inside every database query; other users' records return 404	✓
Logout mechanism	Revokes the session on the server; outstanding tokens stop working immediately	✓
B · Project management		
Create, view, edit and delete projects	Full CRUD API and UI (dialogs, cards, project page); deleting cascades to tasks	✓
Name, description and creation date	Stored and displayed on cards and the project banner	✓
Task counts and completion progress per project	Computed with two grouped SQL queries per page (no N+1)	✓
Prevent access to other users' projects	Scoped queries; covered by cross-user isolation tests	✓
C · Task management		
Create, edit, view and delete tasks within projects	CRUD API; create dialog, side panel, inline quick-add on the board	✓
Title, description, status, priority, due date	Validated on the API (strict ISO dates) and in the forms	✓
Statuses To Do / In Progress / Done; priorities Low / Medium / High	PostgreSQL enums	✓
Change status; filter by status or priority	Drag-and-drop, checkbox, menu, list select; multi-value filters	✓
D · Dashboard and search		
Total projects, total tasks, completed tasks, overdue tasks	`GET /dashboard`, all counts from SQL in one consistent transaction	✓
Search tasks by title combined with status and priority filters	Case-insensitive search backed by a trigram index, combined with all filters	✓
Useful empty state when nothing matches	Shows the search term and a "Clear filters" action	✓

Optional enhancements (brief §6)

ENHANCEMENT	DELIVERED	
Drag-and-drop Kanban board	Yes, with grouping by status, priority or project	✓
Pagination and sorting	Projects and tasks, with stable ordering	✓
Refresh-token rotation / advanced sessions	Rotation, reuse detection, immediate logout	✓
Rate limiting and additional security	Stricter limits on credential endpoints, security headers, CORS allowlist	✓
More tests · loading, error and empty states	116 tests plus an API smoke test in CI · states throughout the app	✓
Deployment to a public hosting platform	Not deployed; one-command Docker setup instead	–

04 · REQUIREMENTS COVERAGE

Technical and API requirements

Brief §3 technical, §4 API capabilities and §5 database & engineering expectations.

AREA	HOW IT IS MET	
Frontend	Next.js + React; responsive layouts; reusable UI components (buttons, fields, dialogs, menus, badges, empty and error states)	✓
Backend	NestJS feature modules: auth, projects, tasks, dashboard, health	✓
API design	REST with correct methods and status codes (200, 201, 204, 400, 401, 404, 409, 429); strict validation rejects unknown or invalid fields	✓
Database	PostgreSQL with foreign keys, cascades, enums and CHECK constraints	✓
State management	TanStack Query + Context + URL state (justified in the architecture note)	✓
Authentication	bcrypt hashing; protected routes on client and server; server-side authorization	✓
Error handling	One JSON error format for every failure, with field-level details and a request ID that also appears in server logs	✓
Git	Meaningful, conventional commits and a detailed README	✓
Testing	Unit and integration tests for the important paths (§6)	✓
Docker	Multi-stage Dockerfiles (non-root) and Docker Compose with health checks	✓
CI/CD	GitHub Actions: lint, type checks, tests, builds and a full-stack smoke test	✓

REST API

METHOD & PATH	PURPOSE
`/api/auth/register · login · refresh · logout · me`	Create an account, log in (httpOnly session cookies + access token), rotate the session, log out on the server, current user
`GET · POST /api/projects`	List (search, sort, pagination; includes task counts and progress) / create
`GET · PATCH · DELETE /api/projects/:id`	Retrieve / update / delete a project (tasks deleted with it)
`GET · POST /api/tasks`	List with `status`, `priority`, `search`, `projectId`, `overdue`, sort, pagination / create
`GET · PATCH · DELETE /api/tasks/:id`	Retrieve / update any field (e.g. only `status`) / delete
`GET /api/dashboard`	Totals, completion rate, overdue and due-today counts, project progress, daily completions, open tasks by priority

> **Security requirement:** ownership is enforced on the server inside every project and task query. Another user's data returns **404** (not 403), so IDs cannot be probed, and moving a task re-checks the destination project. Integration tests verify this with two real users.

Database and engineering expectations

Validation

Invalid or missing fields rejected by the API, backed by database constraints.

Indexes & queries

Owner, status and due-date indexes; trigram search index; grouped statistics.

Correct statistics

Overdue = not done and due before today in the user's timezone; tested.

Missing records and invalid requests return consistent 404 / 400 responses. Secrets come from environment variables, validated at start-up, and none are committed.

05 · ARCHITECTURE

Design and key decisions

A summary of `docs/ARCHITECTURE.md`.

Database design

users	
id	uuid PK
email	unique
name	
password_hash	bcrypt
created_at	

1:n owns

projects	
id	uuid PK
user_id	FK
name · description	
color	
created_at · updated_at	

1:n contains

tasks	
id	uuid PK
project_id	FK
title · description	
status · priority	enums
due_date	date
completed_at	

A `sessions` table (one row per login) stores only a hash of each refresh token. Tasks have no owner column: ownership is derived through their project, so a task can never disagree with its project's owner. `due_date` is a calendar date, avoiding timezone ambiguity for "due on the 18th".

Authentication approach

- **Short-lived access token** (15-minute JWT) plus a **refresh token** that rotates on every use, both in httpOnly cookies that page scripts cannot read.
- **Reuse detection:** presenting an already-used refresh token is treated as theft and the whole session is revoked.
- **Immediate logout:** each access token is tied to its session, so logging out invalidates it at once instead of waiting for it to expire.
- **Hardening:** constant-time login (does not reveal which emails exist), stricter rate limits on credential endpoints, security headers, CORS allowlist.

Key technical decisions

One error format, traceable

Every error returns `{statusCode, error, message, details, path, timestamp, requestId}`. The request ID is echoed in a response header and in server logs, so a reported error can be traced quickly.

Statistics in SQL, not in code

Project progress and dashboard numbers use grouped queries inside one transaction, so the totals are always mutually consistent and cheap to compute.

Timezone-correct "overdue"

The browser sends its timezone; the API decides what "today" is for that user. At 1 a.m. in India it is still yesterday in UTC, and tasks shouldn't flip to overdue early.

Tests use the real pipeline

Integration tests boot the same app setup as production (validation, guards, error filter) against a real PostgreSQL database, not mocks.

Trade-offs and known limitations

- **Not deployed publicly**: the stack is containerised and runs with one command. A cross-domain deployment would add CSRF tokens and `SameSite=None` cookies.
- **Multi-tab refresh race:** two tabs refreshing at once would trip reuse detection; a 30-second grace window for the just-rotated token handles this safely.
- **Rate limits are per instance** (in memory); multiple API replicas would need a shared store such as Redis.
- **Board loads up to 200 tasks per view**; beyond that it points to filters or the paginated list view. Cards are not manually reorderable within a column.
- Single-user workspaces: teams, sharing and password reset are outside the brief's scope.

06 · QUALITY

Testing and continuous integration

116 automated tests, all passing locally and on GitHub Actions.

SUITE	TESTS	WHAT IT COVERS
Backend unit	40	Refresh-token rotation and reuse detection, timing-safe login, ownership checks, completion tracking, timezone-aware dates, error format, environment validation
Backend integration	31	Runs against PostgreSQL: auth flows and cookie flags, logout invalidation, **cross-user isolation**, validation errors, combined filters and search, overdue logic, pagination, cascade delete, exact dashboard numbers across timezones
Frontend	45	API client (single shared token refresh, error mapping), Kanban drag-and-drop for each grouping, quick-add, collapse, complete toggle, theme, command palette, redirect safety, accessible fields
API smoke test	17 req.	Postman collection run with Newman against the full Docker stack; every request asserts success

GitHub Actions pipeline

JOB	STEPS	RESULT
Backend	Install → generate Prisma client → ESLint → TypeScript check → build → unit tests → integration tests on a PostgreSQL service	**Passing**
Frontend	Install → ESLint → TypeScript check → tests → production build	**Passing**
Full stack	Start the whole Docker Compose stack (waits for health checks) → web and Swagger respond → Newman API smoke test → logs on failure → tear down	**Passing**

How this maps to the evaluation scorecard (brief §8)

CATEGORY	EVIDENCE
Frontend: Next.js / React	Reusable component library, responsive layouts, TanStack Query + Context, skeleton / error / empty states, accessible dialogs
Backend: Node.js / NestJS	Modular controllers and services, strict validation, consistent error handling, request logging
APIs and authentication	Correct REST semantics, rotating sessions, server-side ownership checks tested with two users
Database design	Normalised schema, constraints, indexes, migrations, SQL-computed statistics
Testing and code quality	116 tests, lint and type checks enforced in CI, clear separation of concerns
Docker and CI/CD	One-command Compose setup with health checks; three-job GitHub Actions pipeline
Documentation and delivery	README, Swagger, OpenAPI, Postman collection, architecture note, requirements matrix, this document

Repository: github.com/Mithlesh-dhakad1212/Task-Flow
Start here: `README.md` → `docs/REQUIREMENTS.md` → `docs/ARCHITECTURE.md`
Contact: Mithlesh Dhakad · mithleshdhakad89@gmail.com